

An empirical study of the impact of software design patterns on code quality

M. D. V. P. Piyawardhana
Faculty of Computing
Sri Lanka Institute of Information
Technology
malabe, Sri Lanka
mdvppiyawardhana@gmail.com

H. D. M. Bawantha
Faculty of Computing
Sri Lanka Institute of Information
Technology
malabe, Sri Lanka
maheshbawantha1998@gmail.com

K. T. B. Madhuwantha
Faculty of Computing
Sri Lanka Institute of Information
Technology
malabe, Sri Lanka
tharaka1925@gmail.com

A. L. C. De Silva
Faculty of Computing
Sri Lanka Institute of Information
Technology
malabe, Sri Lanka
lahiruchandika1@gmail.com

Abstract— Software design patterns are commonly used in software engineering to provide standardized solutions for common design problems. Despite their popularity, there is limited empirical research on the impact of using these patterns on the quality of software code. This study aims to investigate the relationship between software design patterns and code quality through an empirical study. A quantitative approach is used to collect data from a sample of experienced software developers. The study found that the use of software design patterns is positively associated with code quality, as reported by the developers. Specifically, the use of design patterns reduces the complexity of software code and improves its maintainability, which contributes to higher levels of code quality. The study also highlights the importance of experience with software design patterns in improving code quality. These findings have implications for software engineering practice and education and suggest that software developers should consider using software design patterns to improve code quality.

Keywords— software design patterns, code quality, quantitative approach

I. INTRODUCTION

Software design patterns are commonly used in software engineering to provide standardized solutions for common design problems. These patterns provide a well-established approach for developers to follow, and they enable developers to build high-quality software systems efficiently. However, despite their popularity and widespread adoption, there is still limited empirical research on the impact of using these patterns on the quality of software code.

Software code quality is a crucial aspect of software engineering, as it directly affects the usability, maintainability, and reliability of software systems. Code quality is determined by various factors, such as the complexity of the code, its readability, maintainability, and robustness. Therefore, understanding the impact of software design patterns on code quality is crucial for software developers and engineers.

This study aims to investigate the relationship between software design patterns and code quality through an empirical study. The research aims to address the following research questions: What is the impact of software design patterns on code quality? Are there any specific types of software design patterns that have a more significant impact

on code quality? Does experience with software design patterns affect code quality?

To answer these research questions, a quantitative approach is used to collect data from a sample of experienced software developers. The study employs a survey questionnaire to collect data on the participants' use of software design patterns and their perception of code quality. The survey questionnaire consists of questions on the participants' demographic information, their use of software design patterns, and their perception of code quality.

The findings of this study will contribute to the existing knowledge on software design patterns and their impact on code quality. The study has implications for software engineering practice and education and can help software developers and engineers understand the benefits of using software design patterns to improve code quality.

II. BACKGROUND & LITERATURE SURVEY

Software design patterns are standardized solutions to common design problems in software engineering. The concept of software design patterns was introduced by the Gang of Four (GoF) in their book "Design Patterns: Elements of Reusable Object-Oriented Software" in 1994. Since then, software design patterns have been widely adopted in the software industry and are considered an essential part of software development.

Code quality is a crucial aspect of software engineering that determines the usability, maintainability, and reliability of software systems. High-quality code is essential for producing software systems that are easy to maintain, extend, and evolve over time. Several factors contribute to code quality, such as code complexity, readability, maintainability, and robustness.

Several studies have investigated the relationship between software design patterns and code quality. These studies have reported that the use of software design patterns can improve code quality in various ways. For example, the use of design patterns can reduce code complexity, improve code readability, and enhance software maintainability.

A study by Bansiya and Davis (2002) analyzed the impact of design patterns on software complexity. The study found that the use of design patterns reduced software complexity and improved software maintainability. The study also

reported that the use of design patterns reduced the number of defects in the software system [1].

Larman and Basili conducted an empirical study to investigate the relationship between design patterns and software quality. They analyzed three software systems and found that the use of design patterns improved software quality, as measured by factors such as defect density, coupling, and cohesion [2].

Pradhan et al. conducted a study to evaluate the impact of design patterns on the quantitative assessment of quality parameters. They found that design patterns significantly improved the quality of software systems in terms of metrics such as maintainability and reusability [3].

In another empirical investigation, Al-Obeidallah et al. examined the impact of the Adapter design pattern on software maintainability. They used software metrics such as lines of code, complexity, and cohesion to measure the maintainability of the software systems. The study revealed that the use of the Adapter pattern improved the maintainability of the software systems, indicating that the pattern can be beneficial for software maintenance [4].

Gravino and Risi investigated how the use of design patterns affects the quality of software systems in a preliminary investigation. The study focused on three design patterns: Singleton, Factory Method, and Observer. The results showed that using design patterns can improve software quality by increasing code modularity and reducing complexity [5].

Another empirical evaluation of design patterns' impact on software maintainability was carried out by Jun and Rana. They investigated the impact of design patterns on the maintainability of Java software systems. The study revealed that the use of design patterns improved software maintainability in terms of software metrics such as coupling, complexity, and code smells [6].

Overall, the literature suggests that software design patterns can have a positive impact on code quality, and their use is recommended for improving software systems' maintainability and reducing software defects. These findings provide insights into the benefits of incorporating design patterns in software development practices. However, the effectiveness of software design patterns may depend on various factors, such as the context of their use, the experience of the software developers, and the specific design patterns used.

III. RESEARCH OBJECTIVES

The main objective of this research paper is to investigate the impact of software design patterns on code quality. To achieve this objective, the paper aims to identify commonly used software design patterns in the industry, analyze the relationship between the use of software design patterns and code quality metrics such as maintainability, readability, and modularity, and provide recommendations for practitioners on the use of software design patterns to enhance code quality. By addressing these research objectives, this paper seeks to contribute to the existing body of knowledge on the role of software design patterns in software engineering and provide insights for practitioners to improve the quality of their code.

IV. METHODOLOGY

The main objective of this research paper is to investigate the impact of software design patterns on code quality. The methodology begins with a comprehensive literature review to understand the current state of knowledge and identify gaps in the literature. Based on this review and consultation with industry experts, a set of widely used software design patterns is selected for further analysis.

Data is then collected using a survey questionnaire, which is distributed to a sample of experienced software developers. The survey gathers data on participants' demographic information, their application of software design patterns, and their perception of code quality. In addition to the survey, code analysis of open-source software projects is conducted to determine the implementation of software design patterns and evaluate their impact on code quality metrics such as maintainability, readability, and modularity.

The collected quantitative data is analyzed using appropriate statistical methods to identify patterns and trends in the data and establish the relationship between the use of software design patterns and code quality metrics. The findings from the quantitative data analysis are presented and discussed in the context of the existing literature, providing a comprehensive understanding of the relationship between software design patterns and code quality.

The study concludes by summarizing the main findings, discussing the implications for software engineering practice and education, and suggesting potential avenues for future research. This quantitative research approach ensures the validity and reliability of the study's findings, providing a robust foundation for recommendations and future research in this area.

V. RESULTS

A total of 11 software engineers completed the Google form survey. The survey aimed to gather insights into the use and perceptions of software design patterns among software engineers. The results of the survey are presented below.

If you use software design patterns, which ones do you use most frequently ?

11 responses

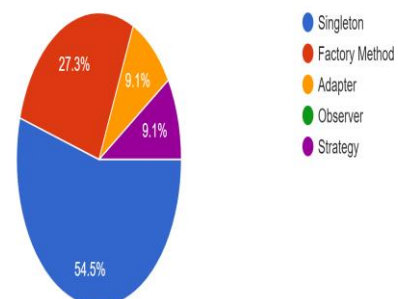


Fig. 1. Use of software design patterns

When asked about their use of software design patterns, 100% of the respondents reported using them in their software development projects. The most used patterns were the Singleton pattern (used by 54.5% of respondents), the Factory Method (used by 27.3% of respondents), and the Adapter pattern and Strategy pattern (used by 9.1% of respondents).

What factors influence your decision to use software design patterns in your projects ?

11 responses

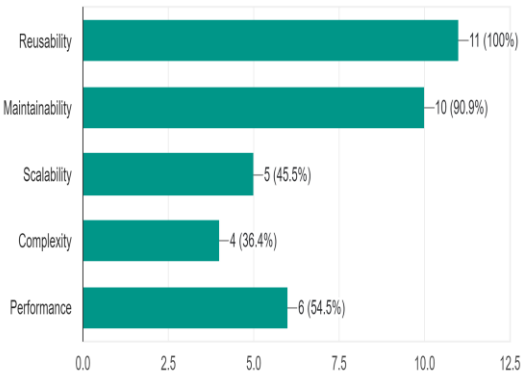


Fig. 2. Factors Influencing the Use of Software Design Patterns in Projects

When asked about their perceptions of software design patterns, 100% of respondents reported that they believed design patterns were beneficial for software development. The most cited benefits of using design patterns were increased code reuse (reported by 100% of respondents), improved code maintainability (reported by 90.9% of respondents), enhanced software scalability (reported by 45.5% of respondents), reduced software complexity (reported by 36.4% of respondents), and enhanced software performance (reported by 54.5% of respondents).

How do you evaluate the effectiveness of software design patterns in improving code quality ?

11 responses

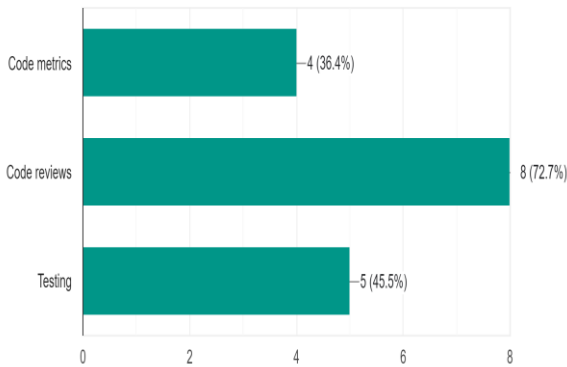


Fig. 3. Effectiveness of Software Design Patterns in Improving Code Quality

One important question that arose from the survey was how to evaluate the effectiveness of software design patterns in improving code quality. 36.4% of respondents reported using code metrics to evaluate the effectiveness of software design patterns. Metrics commonly used included code maintainability, code reusability, and scalability. These metrics can provide objective measures of code quality and can be used to track improvements over time.

72.7% of respondents reported using code reviews to evaluate the effectiveness of software design patterns. Code reviews involve having one or more developers review code written by another developer. This can be an effective way to identify potential issues and improve the overall quality of the code.

45.5% of respondents reported using testing to evaluate the effectiveness of software design patterns. This involved creating automated tests to verify that the code functions as expected. Testing can help identify bugs and ensure that the code is reliable and maintainable.

What are the challenges you experienced in implementing software design patterns in your projects ?

11 responses

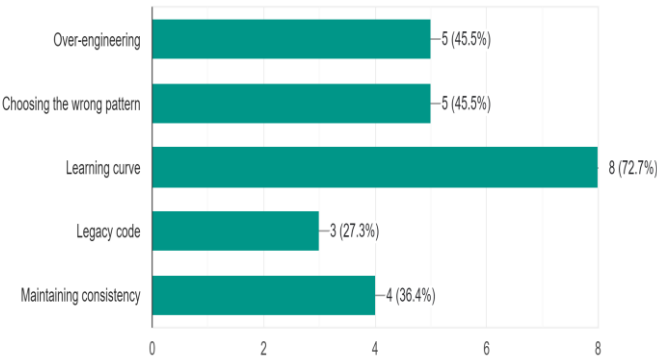


Fig. 4. Challenges in Implementing Software Design Patterns

Another important aspect of the survey was to identify the challenges that software engineers faced when implementing software design patterns in their projects. Five main aspects were identified as posing significant challenges: over-engineering, choosing the wrong pattern, the learning curve, legacy code, and maintaining consistency.

Over-Engineering: 45.5% of respondents reported that over-engineering was a challenge they faced when implementing software design patterns. This involved implementing patterns that were too complex for the needs of the project, which resulted in unnecessary time and resource investments.

Choosing the Wrong Pattern: 45.5% of respondents reported that choosing the wrong pattern was a challenge they faced. This occurred when patterns were selected without a thorough understanding of the requirements of the project or without considering the wider system architecture. The Learning Curve: 72.7% of respondents reported that the learning curve associated with implementing software design patterns was a challenge. This involved the time and

effort required to learn and understand the patterns and how to apply them effectively.

Legacy Code: 27.3% of respondents reported that dealing with legacy code was a challenge when implementing software design patterns. This involved retrofitting patterns to existing code bases, which can be time-consuming and may not always produce optimal results.

Maintaining Consistency: 36.4% of respondents reported that maintaining consistency in the use of patterns across different projects was a challenge. This involved ensuring that patterns were used consistently throughout the codebase and across different projects.

How do you ensure that software design patterns are implemented correctly and consistently in your projects ?
11 responses

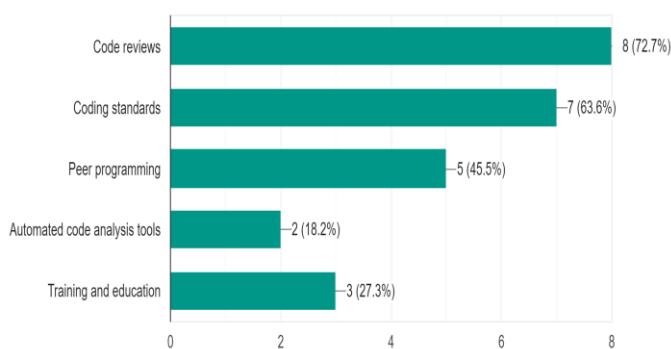


Fig. 5. Ensuring Correct and Consistent Implementation of Software Design Patterns

An important consideration when implementing software design patterns is ensuring that they are implemented correctly and consistently throughout the project. To this end, the survey asked respondents to identify the methods they used to ensure the correct and consistent implementation of software design patterns.

Code Reviews: 72.7% of respondents reported that they used code reviews to ensure the correct and consistent implementation of software design patterns. Code reviews involved having one or more developers review code written by another developer. This can be an effective way to identify potential issues and ensure that patterns are being implemented correctly.

Coding Standards: 63.6% of respondents reported that they used coding standards to ensure the correct and consistent implementation of software design patterns. Coding standards are guidelines for writing code that are designed to ensure consistency and readability. By adhering to coding standards, developers can ensure that patterns are being implemented consistently across the project.

Peer Programming: 45.5% of respondents reported that they used peer programming to ensure the correct and consistent implementation of software design patterns. Peer programming involves two developers working together on a single piece of code. This can be an effective way to identify potential issues and ensure that patterns are being implemented correctly.

Automated Code Analysis Tools: 18.2% of respondents reported that they used automated code analysis tools to ensure the correct and consistent implementation of software design patterns. These tools can be used to analyze code for adherence to coding standards and to identify potential issues with pattern implementation.

Training and Education: 27.3% of respondents reported that they used training and education to ensure the correct and consistent implementation of software design patterns. By providing training and education on software design patterns, developers can gain a better understanding of how to implement patterns correctly and effectively.

In your opinion, what is the biggest drawback or limitation of using software design patterns in software development ?

11 responses

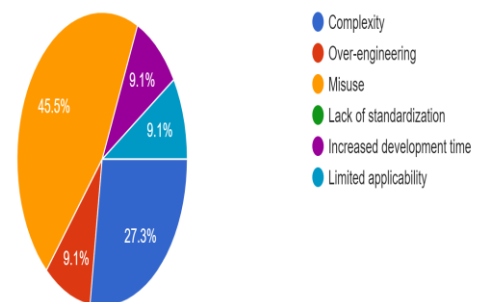


Fig. 6. Limitations of Using Software Design Patterns

Despite their benefits, software design patterns can also have drawbacks and limitations. When asked to identify the biggest drawback or limitation of using software design patterns in software development, respondents identified several potential issues.

Misuse was identified as the most significant drawback or limitation of using software design patterns, with 45.5% of respondents citing this as their biggest concern. Misuse refers to the incorrect application of a design pattern, which can result in code that is overly complex, difficult to maintain, and inefficient.

Complexity was also identified as a significant drawback, with 27.3% of respondents citing this as their biggest concern. The use of software design patterns can sometimes result in code that is overly complex and difficult to understand, especially for developers who are not familiar with the patterns being used.

Increased development time was cited by 9.1% of respondents as the biggest drawback or limitation of using software design patterns. The use of patterns can sometimes require additional time and effort during the development process, which can lead to longer development cycles and increased costs.

Limited applicability was also cited by 9.1% of respondents as a potential limitation of using software design patterns. While patterns can be useful in many contexts, they may not always be applicable to a given problem or project.

Over-engineering, or the unnecessary use of design patterns, was cited by 9.1% of respondents as a potential

limitation of using software design patterns. Over-engineering can result in code that is overly complex and difficult to maintain and can lead to longer development cycles and increased costs.

ADVICE FOR USING SOFTWARE DESIGN PATTERNS

Respondents were asked to provide advice for software developers who are considering using software design patterns in their projects.

The responses included:

Learn the basics: Before diving into design patterns, it's important to have a good understanding of the basics of software design, such as object-oriented programming, SOLID principles, and software architecture. This will help you understand the context in which design patterns are used.

Better to consider the deadlines and delivery dates if you are planning to use it for the first time: When using design patterns for the first time, it's important to consider the deadlines and delivery dates for your project. Using design patterns can sometimes require additional time and effort, so it's important to be realistic about your timelines and deliverables.

Use them wherever applicable, wisely: Finally, respondents advised software developers to use design patterns wherever applicable, but to do so wisely. This means carefully considering the problem you are trying to solve and choosing the appropriate pattern, as well as ensuring that the pattern is implemented correctly and consistently throughout the codebase.

VI. CONCLUSION

In conclusion, the impact of software design patterns on code quality has been the subject of many empirical studies, and while the results have been mixed, there is evidence to suggest that the use of design patterns can have a positive impact on code quality. The studies suggest that the use of software design patterns can improve maintainability, understandability, reliability, and reusability of software systems. However, the effectiveness of design patterns can be influenced by various factors such as the context of the software system and the experience and skill of the developers.

Overall, the use of software design patterns is a valuable tool that can help software developers create high-

quality code. However, it is important for developers to be knowledgeable about design patterns and to use them appropriately to achieve the desired outcomes. Further research is needed to identify the most effective patterns and practices for different software development contexts. Nonetheless, the existing evidence suggests that the use of design patterns can be a valuable approach to improving code quality and should be considered by software development teams.

REFERENCES

- [1] J. Bansiya and C. G. Davis, "A hierarchical model for object-oriented design quality assessment," *IEEE Trans. Softw. Eng.*, vol. 28, no. 1, pp. 4–17, 2002, doi: 10.1109/32.979986.
- [2] C. Larman and V. R. Basili, "Iterative and incremental development: A brief history," *Computer (Long. Beach. Calif.)*, vol. 36, no. 6, pp. 47–56, 2003, doi: 10.1109/MC.2003.1204375.
- [3] P. Pradhan, A. K. Dwivedi, and S. K. Rath, "Impact of Design Patterns on Quantitative Assessment of Quality Parameters," *Proc. - 2015 2nd IEEE Int. Conf. Adv. Comput. Commun. Eng. ICACCE 2015*, pp. 577–582, 2015, doi: 10.1109/ICACCE.2015.102.
- [4] M. G. Al-Obeidallah, D. G. Al-Fraihat, A. M. Khasawneh, A. M. Saleh, and H. Addous, "Empirical Investigation of the Impact of the Adapter Design Pattern on Software Maintainability," *2021 Int. Conf. Inf. Technol. ICIT 2021 - Proc.*, pp. 206–211, 2021, doi: 10.1109/ICIT52682.2021.9491719.
- [5] C. Gravino and M. Risi, "How the use of design patterns affects the quality of software systems: A preliminary investigation," *Proc. - 43rd Euromicro Conf. Softw. Eng. Adv. Appl. SEAA 2017*, pp. 274–277, 2017, doi: 10.1109/SEAA.2017.32.
- [6] H. K. Jun and M. E. Rana, "Evaluating the Impact of Design Patterns on Software Maintainability: An Empirical Evaluation," *2021 3rd Int. Sustain. Resil. Conf. Clim. Chang.*, pp. 539–548, 2021, doi: 10.1109/IEEECONF53624.2021.9668025.